PostgreSQL 18 Accidental TRUNCATE Recovery

Production Technical SOP with pgBackRest Point-in-Time Recovery (PITR)

Committed and Uncommitted TRUNCATE | Backup-Only Recovery | No-Backup Scenario | 100-GB Table Recovery Lab

Document Type	Technical SOP / DBA Recovery Runbook
Platform	PostgreSQL 18.x
Backup Tool	pgBackRest
Primary Use Case	Developer accidentally truncates a production table
Recovery Pattern	Separate PITR recovery instance + table-level reconciliation
Validation Basis	PostgreSQL core behavior and pgBackRest recovery capabilities

Production safety principle: preserve the current production database. Recover historical data on a separate server whenever practical, then reconcile only the required rows back into production.

Contents / Index

Section	Topic	Page
1	Purpose and Scope	3
2	Incident Scenario and Key Facts	3
3	Recovery Decision Flow	3
4	Immediate Incident Response	4
5	TRUNCATE Not Yet Committed	4
6	TRUNCATE Already Committed	5
7	PITR Is Available	5
8	Backup Exists but PITR/WAL Does Not	5
9	No Backup and No Usable PITR	6
10	PostgreSQL 18 + pgBackRest PITR Recovery	6
11	CASCADE and RESTART IDENTITY Considerations	10
12	Full PostgreSQL 18 Practical Lab	10
13	100-GB Table Recovery Considerations	12
14	Why a Separate Recovery Server Is Safer	13
15	Prevention and Security Controls	13
16	Production DBA Checklists	14
17	Incident / RCA Template	15
18	Recommended Enterprise Recovery Pattern	15
19	Official References	16

1. Purpose and Scope

- This SOP explains how a PostgreSQL DBA should respond when a developer accidentally executes TRUNCATE TABLE against a production table.
- It covers uncommitted TRUNCATE, committed TRUNCATE, PITR recovery, backup-only recovery, and the case where neither backup nor usable PITR exists.
- The main recovery pattern uses PostgreSQL physical backup + WAL replay through pgBackRest to recover a separate instance to a point immediately before the destructive transaction.
- The document also includes a practical PostgreSQL 18 lab and special considerations for a very large table such as a 100-GB production object.

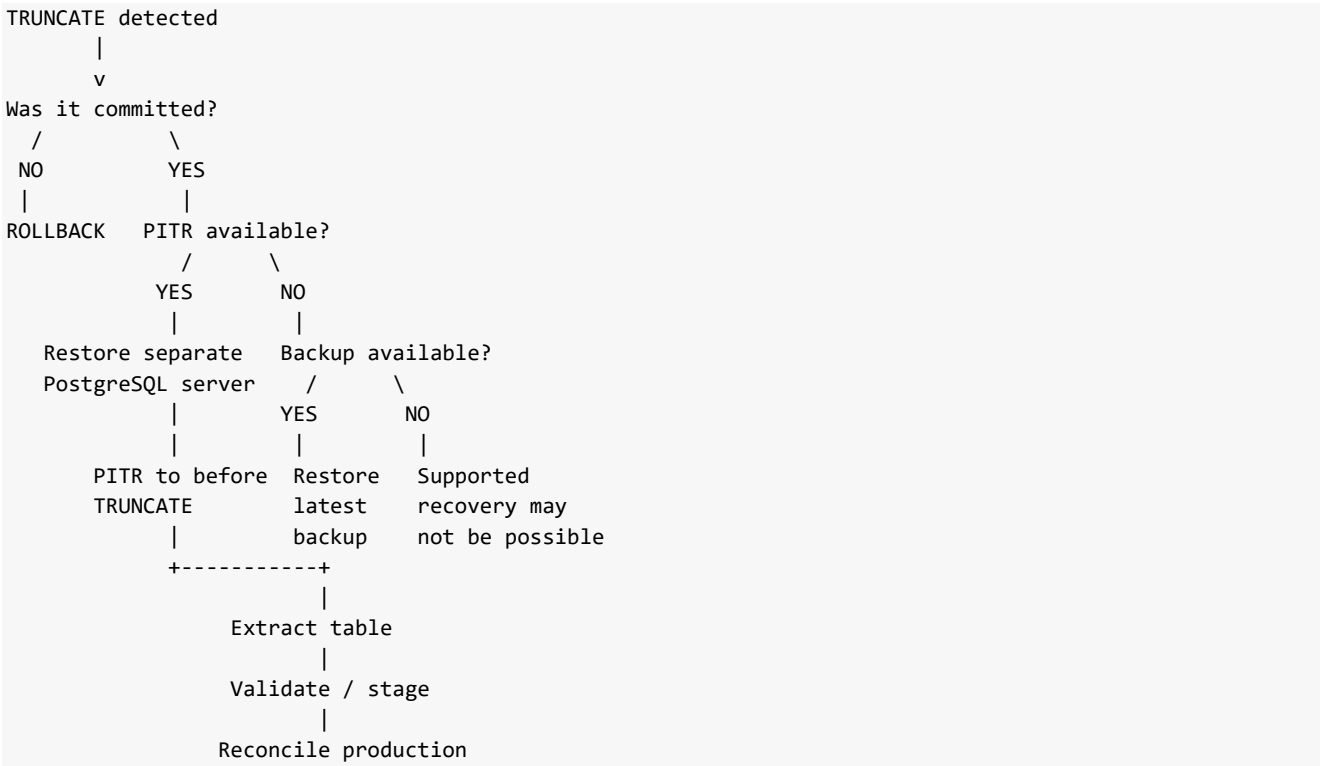
2. Incident Scenario and Key Facts

Item	Example
Database	prod_db
Schema / Table	public.customer_orders
Table Size	Approximately 100 GB
PostgreSQL	18.x
Backup Tool	pgBackRest
Incident	Accidental TRUNCATE issued by developer
Primary Question	Was the TRUNCATE transaction committed?

```
TRUNCATE TABLE public.customer_orders;
```

The recovery decision starts with transaction state. PostgreSQL TRUNCATE is transactional: if it has not committed, ROLLBACK can restore the rows. After commit, ROLLBACK cannot undo it.

3. Recovery Decision Flow



4. Immediate Incident Response

- Declare the incident and identify the exact database, schema, table, application, user, session, and approximate execution time.
- Determine whether the TRUNCATE is still part of an open transaction or has already committed.
- Check whether CASCADE or RESTART IDENTITY was used. CASCADE can truncate dependent tables; RESTART IDENTITY affects owned sequences.
- Preserve the current production state. Do not immediately restore the entire production cluster backward.
- Capture PostgreSQL server time and timezone before selecting a PITR target.
- Preserve relevant PostgreSQL, pgAudit, application, connection-pool, SIEM, and change-ticket evidence.

```
SELECT now();
SHOW timezone;
SELECT COUNT(*) FROM public.customer_orders;
SELECT pg_size_pretty(pg_total_relation_size('public.customer_orders'));
```

5. Scenario 1 - TRUNCATE Not Yet Committed

If the developer started a transaction and TRUNCATE has not committed, rollback is the cleanest recovery path.

```
BEGIN;
TRUNCATE TABLE public.customer_orders;
-- Accident detected before COMMIT
ROLLBACK;

SELECT COUNT(*) FROM public.customer_orders;
```

- PostgreSQL treats TRUNCATE as transaction-safe with respect to rollback of the surrounding transaction.
- Do not issue COMMIT after discovering the error.
- Coordinate with the session owner or DBA before terminating sessions blindly; a terminated backend generally aborts its active transaction, but incident handling should still be controlled.
- Validate row count, application access, sequence state if relevant, and dependent tables.

6. Scenario 2 - TRUNCATE Already Committed

```
BEGIN;
TRUNCATE TABLE public.customer_orders;
COMMIT;
```

After commit, a later ROLLBACK cannot restore the rows. The normal enterprise recovery options are physical backup + archived WAL for PITR, or restoration from the most recent usable backup if PITR is unavailable.

Do not assume the presence of files in pg_wal means PostgreSQL provides a supported table-level UNDO TRUNCATE function. It does not.

7. Scenario 3 - PITR Is Available

Time	Event
14:00:00	Production healthy
14:05:13	Developer executes TRUNCATE
14:05:14	Transaction commits
14:06:00	Monitoring detects zero/abnormal rows
14:10:00	DBA begins recovery

A typical target is immediately before the destructive transaction or, even better, a known recovery boundary such as a named restore point created before a controlled operation.

```
recovery target example: 2026-09-09 14:05:12-05
```

1. Restore a separate PostgreSQL instance from a suitable physical backup.
2. Replay the required archived WAL up to the recovery target.
3. Validate the recovered table on the isolated instance.
4. Export the required table or rows.
5. Load recovered data to a staging location and reconcile it with current production data.
6. Keep legitimate post-incident production transactions rather than rolling the entire production database backward.

8. Scenario 4 - Backup Exists but PITR/WAL Does Not

If a recent backup exists but the WAL required to reach the incident time is unavailable, restore the latest valid backup to a separate server and recover whatever data is represented by that backup.

Example	Value
Last usable backup	Tuesday 23:00
TRUNCATE	Wednesday 14:05
Potential data gap	Approximately 15 hours

- Rows inserted after the last backup may be missing from the restored copy.
- Updates and deletes made after the backup will not be represented accurately.
- The DBA should quantify the data gap and obtain application/business validation before reinserting historical rows.

9. Scenario 5 - No Backup and No Usable PITR

- If the TRUNCATE committed and there is no suitable backup, no archived WAL chain for PITR, no replica or other trusted copy, and no application-level source of truth, normal supported recovery may not be possible.
- PostgreSQL does not provide a built-in UNDO TRUNCATE command for an already committed transaction.
- Forensic recovery attempts from storage/WAL internals are specialized, risky, and should not be treated as a standard recovery procedure.
- Escalate the incident, preserve storage, logs, and infrastructure snapshots where applicable, and avoid additional destructive maintenance while specialized recovery options are assessed.

This is why tested backups, WAL archiving, restore drills, access control, and audit logging are mandatory production controls.

10. Scenario 6 - PostgreSQL 18 + pgBackRest PITR Recovery

Component	Example
Production server	pgsql01
Recovery server	pgsql-recovery01
Database	prod_db
Table	public.customer_orders

Component	Example
Size	~100 GB
Stanza	pg18
Repository	/DBBackups2026/pgbackrest
Incident time	2026-09-09 14:05:13-05
Target time	2026-09-09 14:05:12-05

Step 1 - Record Incident Details

- Database, schema, table, application, user/developer, SQL statement, approximate execution time, and whether it committed.
- Record whether CASCADE or RESTART IDENTITY was used.

Step 2 - Preserve Production

- Do not restore over production as the first response.
- Avoid unnecessary VACUUM FULL, schema rewrites, table recreation, or other disruptive activity until the recovery plan is agreed.

Step 3 - Validate the Affected Object

```
SELECT COUNT(*) FROM public.customer_orders;
SELECT pg_size_pretty(pg_total_relation_size('public.customer_orders'));
```

Step 4 - Confirm Server Time and Timezone

- Use an explicit timezone offset in the recovery target to avoid ambiguity.

```
SELECT now();
SHOW timezone;
```

Step 5 - Determine the Exact TRUNCATE/Commit Boundary

- Use PostgreSQL logs, pgAudit, application logs, SIEM, connection-pool logs, shell history, and change records.
- The transaction commit boundary is more important than simply subtracting an arbitrary second from the statement timestamp.

```
grep -i "truncate" /var/log/postgresql/postgresql-*.log
```

Step 6 - Validate pgBackRest

```
sudo -u postgres pgbackrest --stanza=pg18 check
sudo -u postgres pgbackrest --stanza=pg18 info
```

Step 7 - Confirm WAL Coverage

- Ensure the selected backup plus archived WAL can reach the requested target time.
- Any gap in required WAL can prevent successful PITR.

Step 8 - Prepare an Isolated Recovery Host

- Use a separate IP/DNS name and restrict application connectivity.
- Use a different port if useful, for example production 5432 and recovery 5433.

Step 9 - Confirm the Recovery Host Data Directory

```
sudo -u postgres psql -c "SHOW data_directory;"
```

Step 10 - Stop PostgreSQL on the Recovery Host

```
sudo systemctl stop postgresql
sudo systemctl status postgresql --no-pager
```

Step 11 - Protect Any Existing Recovery-Host Data

- If the recovery host already contains a cluster, move or otherwise preserve it before restore. Never perform this step against the live production data directory.

```
sudo mv /var/lib/postgresql/18/main /var/lib/postgresql/18/main_before_restore
```

Step 12 - Identify a Suitable Backup Set

```
sudo -u postgres pgbackrest --stanza=pg18 info
```

Step 13 - Restore to the PITR Target

- For a recovery exercise, target-action=pause offers additional control; target-action=promote can be used when immediate promotion is desired.
- Use --set only when you intentionally want to force a specific backup set.

```
sudo -u postgres pgbackrest \
  --stanza=pg18 \
  --type=time \
  --target="2026-09-09 14:05:12-05" \
  --target-action=pause \
  restore
```

Step 14 - Start PostgreSQL and Monitor Recovery

```
sudo systemctl start postgresql
sudo journalctl -u postgresql -f
```

Step 15 - Verify Recovery State

```
SELECT pg_is_in_recovery();
```

Step 16 - Validate the Recovered Table

```
SELECT COUNT(*) FROM public.customer_orders;
SELECT pg_size_pretty(pg_total_relation_size('public.customer_orders'));
```

Step 17 - Perform Business-Level Validation

```
SELECT MIN(order_date), MAX(order_date), COUNT(*), SUM(order_amount)
FROM public.customer_orders;

SELECT * FROM public.customer_orders
ORDER BY order_date DESC
LIMIT 100;
```

Step 18 - Check Table Definition and Dependencies

- Validate primary keys, foreign keys, unique constraints, check constraints, defaults, indexes, sequences, triggers, and partition relationships.

```
\d+ public.customer_orders
```

Step 19 - Export the Recovered Table

```
pg_dump -h localhost -p 5432 -U postgres -d prod_db \
  -t public.customer_orders -Fc \
  -f /DBBackups2026/customer_orders_recovered.dump
```

Step 20 - Restore into Staging, Not Blindly Over Production

- Prefer an isolated recovery database/schema or controlled staging object for reconciliation.

```
CREATE SCHEMA recovery_20260909;

CREATE TABLE recovery_20260909.customer_orders
(LIKE public.customer_orders INCLUDING ALL);
```

Step 21 - Compare Current and Recovered Data

```
SELECT COUNT(*) FROM public.customer_orders;
SELECT COUNT(*) FROM recovery_20260909.customer_orders;

SELECT COUNT(*)
FROM recovery_20260909.customer_orders r
LEFT JOIN public.customer_orders p ON p.order_id = r.order_id
WHERE p.order_id IS NULL;
```

Step 22 - Reinsert Only Missing Rows After Validation

- For a 100-GB table, consider controlled batches to limit transaction size, WAL generation, replica lag, and lock duration.

```
INSERT INTO public.customer_orders
SELECT r.*
FROM recovery_20260909.customer_orders r
WHERE NOT EXISTS (
  SELECT 1 FROM public.customer_orders p
  WHERE p.order_id = r.order_id
);
```

Step 23 - Compare Changed Rows Separately

- Do not overwrite newer production values automatically. Resolve conflicts using application/business rules.

```
SELECT p.order_id,
       p.order_status AS current_status,
       r.order_status AS recovered_status
FROM public.customer_orders p
JOIN recovery_20260909.customer_orders r USING (order_id)
WHERE p.order_status IS DISTINCT FROM r.order_status;
```

Step 24 - Validate Sequence / Identity State

- Only reset sequence state when the table design and recovery process require it.

```
SELECT MAX(order_id) FROM public.customer_orders;
```

Step 25 - Refresh Statistics

```
ANALYZE public.customer_orders;
```

Step 26 - Validate Index State

```
SELECT indexrelid::regclass, indisvalid, indisready
FROM pg_index
WHERE indrelid='public.customer_orders'::regclass;
```

Step 27 - Validate Replication

```
SELECT application_name, client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn
FROM pg_stat_replication;
```

Step 28 - Obtain Application and Business Sign-Off

- Test critical SELECT/INSERT/UPDATE/DELETE paths, APIs, batch jobs, reporting, and referential-integrity operations before incident closure.

Step 29 - Close the Incident and Document RCA

- Record incident timestamps, backup set, WAL coverage, recovery target, pgBackRest commands, rows recovered, reconciled conflicts, achieved RPO/RTO, root cause, and preventive actions.

11. CASCADE and RESTART IDENTITY Considerations

11.1 TRUNCATE ... CASCADE

```
TRUNCATE TABLE public.customer_orders CASCADE;
```

- CASCADE can truncate tables that have foreign-key references to the target table.
- Identify every affected relation before performing table-level recovery.
- Recovering only the initially named table may leave the database inconsistent if dependent tables were also truncated.

11.2 TRUNCATE ... RESTART IDENTITY

```
TRUNCATE TABLE public.customer_orders RESTART IDENTITY;
```

- Owned sequences are restarted as part of the TRUNCATE operation.
- If the transaction is rolled back, the sequence restart is transactionally rolled back with it.
- After PITR/table-level reconciliation, verify sequence/identity state so that new inserts cannot collide with restored keys.

12. Full PostgreSQL 18 Practical Lab

12.1 Create the Lab Database and Table

```
CREATE DATABASE truncate_lab;

CREATE TABLE customer_orders
(
    order_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id bigint NOT NULL,
    order_date timestamp NOT NULL DEFAULT now(),
    order_amount numeric(12,2),
    order_status varchar(30)
);
```

12.2 Load One Million Sample Rows

```
INSERT INTO customer_orders(customer_id, order_amount, order_status)
SELECT
    (random() * 100000)::bigint,
    (random() * 5000)::numeric(12,2),
    CASE
      WHEN random() < 0.25 THEN 'NEW'
      WHEN random() < 0.50 THEN 'PROCESSING'
      WHEN random() < 0.75 THEN 'SHIPPED'
      ELSE 'DELIVERED'
```

```
    END
FROM generate_series(1,1000000);

SELECT COUNT(*) FROM customer_orders;
```

12.3 Example pgBackRest Configuration

```
[global]
repo1-path=/DBBackups2026/pgbackrest
repo1-retention-full=2
start-fast=y
process-max=4
log-level-console=info

[pg18]
pg1-path=/var/lib/postgresql/18/main

archive_mode = on
archive_command = 'pgbackrest --stanza=pg18 archive-push %p'
```

Paths and service names vary by operating system/package. Always confirm data_directory and pgBackRest repository paths on the actual host.

12.4 Create and Validate the Stanza

```
sudo -u postgres pgbackrest --stanza=pg18 stanza-create
sudo -u postgres pgbackrest --stanza=pg18 check
sudo -u postgres pgbackrest --stanza=pg18 info
```

12.5 Take a Full Backup

```
sudo -u postgres pgbackrest --stanza=pg18 --type=full backup
```

12.6 Capture a Baseline

```
SELECT COUNT(*), MIN(order_id), MAX(order_id), SUM(order_amount) FROM customer_orders;
```

12.7 Create a Named Restore Point

```
SELECT pg_create_restore_point('before_truncate_test');
```

12.8 Simulate the Developer Accident

```
SELECT clock_timestamp();
TRUNCATE TABLE customer_orders;
SELECT COUNT(*) FROM customer_orders;
```

12.9 Generate Post-Incident Activity

```
INSERT INTO customer_orders(customer_id, order_amount, order_status)
SELECT (random()*100000)::bigint,
       (random()*5000)::numeric(12,2),
       'NEW'
FROM generate_series(1,1000);
```

This intentionally creates new rows after the accident and demonstrates why production should not simply be rolled backward.

12.10 Restore a Separate Instance to the Named Point

```
sudo -u postgres pgbackrest \
  --stanza=pg18 \
```

```
  --type=name \
  --target="before_truncate_test" \
  --target-action=promote \
  restore
```

12.11 Validate and Export the Recovered Table

```
SELECT COUNT(*) FROM customer_orders;

pg_dump -d truncate_lab -t customer_orders -Fc -f customer_orders_before_truncate.dump
```

12.12 Reconcile with Production

7. Load recovered data into staging.
8. Identify rows missing from the current production table.
9. Identify rows that exist in both locations but have different current values.
10. Restore only approved historical data.
11. Preserve legitimate post-incident transactions.
12. Validate sequences, statistics, indexes, replicas, application behavior, and business totals.

13. 100-GB Table Recovery Considerations

Area	DBA Concern
Physical restore	The whole PostgreSQL cluster may need to be restored before one table can be extracted.
WAL replay	Large WAL volume can extend recovery time.
pg_dump	A 100-GB object can create significant read I/O and a large dump file.
Network	Moving the recovered dump may take substantial time and bandwidth.
Reload	Reinserting rows creates WAL and can affect checkpoints and replicas.
Indexes	Maintaining many indexes during reload increases I/O and CPU.
Replication	A large recovery insert can cause replica lag.
Disk capacity	Plan for restored cluster + dump + WAL + temporary space + index overhead + safety margin.
Locks	Batching may reduce long transaction and lock exposure.
RTO	Table extraction/reconciliation can dominate total recovery time even after PITR finishes.

For very large tables, capacity planning and reconciliation strategy should be designed before the incident. A successful PITR does not automatically mean the business table is restored to production quickly.

14. Why a Separate Recovery Server Is Safer

```
14:05  TRUNCATE commits
14:06  New orders arrive
14:07  Payments are posted
14:08  Shipments are updated
14:09  Customer changes are saved
14:10  DBA discovers incident
```

Rolling the live cluster back to 14:04:59 would remove legitimate work performed after the incident. The safer pattern is:

```
PITR recovery clone -> extract lost historical data -> staging -> compare -> reconcile -> current
production
```

15. Prevention and Security Controls

15.1 Restrict TRUNCATE Privilege

```
REVOKE TRUNCATE ON TABLE public.customer_orders FROM developer_role;
```

15.2 Use Read-Only Production Roles Where Possible

```
GRANT CONNECT ON DATABASE prod_db TO developer_readonly;
GRANT USAGE ON SCHEMA public TO developer_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO developer_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT ON TABLES TO developer_readonly;
```

15.3 Review Current Grants

```
SELECT grantee, privilege_type
FROM information_schema.role_table_grants
WHERE table_schema='public'
  AND table_name='customer_orders';
```

15.4 Audit and Alert on Destructive Operations

- Alert on TRUNCATE, DROP TABLE, DROP DATABASE, and unexpected production DDL.
- Consider pgAudit or an equivalent enterprise audit approach where required.
- Route alerts to the DBA/on-call system and include database, user, client address, statement, and server-local timestamp where available.
- Perform restore drills regularly; a backup that has never been restored is not a proven recovery solution.

16. Production DBA Checklists

16.1 Before Recovery

- ☐ Confirm database/schema/table
- ☐ Confirm commit state
- ☐ Check CASCADE
- ☐ Check RESTART IDENTITY
- ☐ Determine exact incident/commit time
- ☐ Confirm server timezone
- ☐ Preserve current production
- ☐ Validate pgBackRest stanza
- ☐ Validate usable physical backup
- ☐ Validate continuous WAL coverage
- ☐ Prepare isolated recovery host

16.2 During Recovery

- ☐ Restore to separate host
- ☐ Use correct PITR target
- ☐ Monitor WAL replay
- ☐ Verify recovery target reached
- ☐ Validate row count and business totals

- ☐ Validate dependent tables
- ☐ Validate sequence state
- ☐ Export only required data
- ☐ Load recovered data into staging

16.3 After Recovery

- ☐ Identify missing rows
- ☐ Identify conflicting/changed rows
- ☐ Reconcile approved data
- ☐ Run ANALYZE if needed
- ☐ Validate indexes
- ☐ Check replication lag
- ☐ Validate application
- ☐ Validate business totals
- ☐ Obtain business sign-off
- ☐ Document RCA/RPO/RTO
- ☐ Remove unnecessary privileges
- ☐ Review alerts and backup policy

17. Incident / RCA Template

Field	Value / Notes
Incident ID	
Detection time	
Database / schema / table	
User / application	
Executed statement	
TRUNCATE committed?	
CASCADE / RESTART IDENTITY?	
Exact commit boundary	
Backup selected	
WAL coverage	
Recovery target	
Recovery server	
Rows recovered	
Rows reconciled	
Conflicts identified	
RPO achieved	
RTO achieved	
Business validation	
Root cause	
Preventive actions	

18. Recommended Enterprise Recovery Pattern

```
Preserve production
      |
Identify exact incident/commit boundary
      |
Validate backup + archived WAL
      |
Restore separate PostgreSQL instance
      |
Replay WAL to immediately before TRUNCATE
      |
Validate recovered table and dependencies
      |
Export historical data
      |
Load to staging
      |
Compare with current production
      |
Restore/reconcile only required data
      |
Validate application + business + replication
      |
Close incident and implement prevention controls
```

The objective is not simply to make the old rows visible again. The objective is to recover the lost historical data while preserving valid transactions that occurred after the TRUNCATE.

19. Official References

PostgreSQL 18 - TRUNCATE: https://www.postgresql.org/docs/18/sql-truncate.html

PostgreSQL 18 - Continuous Archiving and Point-in-Time Recovery:
https://www.postgresql.org/docs/18/continuous-archiving.html

PostgreSQL 18 - Recovery Configuration: https://www.postgresql.org/docs/18/runtime-config-wal.html

PostgreSQL 18 - pg_create_restore_point: https://www.postgresql.org/docs/18/functions-admin.html

pgBackRest User Guide: https://pgbackrest.org/user-guide.html

pgBackRest Command Reference: https://pgbackrest.org/command.html